# NON-NUMERIC VALUES

- we'll consider three important non numeric data types: logicals, characters, and factors

```
> x <- c(3,4,5)
> x
[1] 3 4 5
```

Delete

```
> myvec <- c(1,3,1,42)
> myvec
[1]  1  3  1 42
> myvec2 <-c(3,-3,2,5,myvec)
> myvec2
[1]  3 -3  2  5  1  3  1 42
> myvec3 <- c(myvec,myvec2)
> myvec3
 [1]  1  3  1 42  3 -3  2  5  1  3  1 42
```

# LOGICALS, CHARACTERS, AND FACTORS

## Logical Values

- Logical values are based on a simple premise:a logical-valued object can only be either TRUE or FALSE.

```
> satisfied = TRUE
> satisfied
[1] TRUE
> notsatisfied = FALSE
> notsatisfied
[1] FALSE
> review <- c(T,F,F,F,T,F,T,T,T,F,T,F)
> review
 [1]  TRUE FALSE FALSE FALSE  TRUE FALSE  TRUE  TRUE  TRUE FALSE  TRUE
[12] FALSE
> length(x=review)
[1] 12
```

# LOGICALS

## Logical Outcome: Relational Operators

```
> 2==3
[1] FALSE
> 2>3
[1] FALSE
> (3-2)<=3
[1] TRUE
> 2!=(3+4)
[1] TRUE
> foo <- c(3,2,1,4,1,2,1,-1,0,3)
> bar <- c(4,1,2,1,1,0,0,3,0,4)
> length(x=foo)==length(x=bar)
[1] TRUE
> foo==bar
 [1] FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE
```

```
> foo.mat <- matrix(foo,nrow=5,ncol=2)
> foo.mat
     [,1] [,2]
[1,]    3    2
[2,]    2    1
[3,]    1   -1
[4,]    4    0
[5,]    1    3
```

# CLASS EXERCISE

a. Store the following vector of 15 values as an object in your workspace: `c(6,9,7,3,6,7,9,6,3,6,6,7,1,9,1)`. Identify the following elements:
   i. Those equal to 6
   ii. Those greater than or equal to 6
   iii. Those less than 6 + 2
   iv. Those not equal to 6

ST331

(Statistical Computing 3)

# CLASS EXERCISE

```
> v <- c(6, 9, 7, 3, 6, 7, 9, 6, 3, 6, 6, 7, 1, 9, 1)
> v==6
 [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE  TRUE  TRUE FALSE FALSE FALSE FALSE
> v>=6
 [1]  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE FALSE  TRUE FALSE
> v<(6+2)
 [1]  TRUE FALSE  TRUE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE  TRUE
> v!=6
 [1] FALSE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE FALSE  TRUE FALSE FALSE  TRUE  TRUE  TRUE  TRUE
```

ST331

(Statistical Computing 3)

# LOGICALS

## Multiple Comparisons: Logical Operators

| Operator | Interpretation | Results |
|---|---|---|
| & | AND (element-wise) | TRUE & TRUE is TRUE<br>TRUE & FALSE is FALSE<br>FALSE & TRUE is FALSE<br>FALSE & FALSE is FALSE |
| && | AND (single comparison) | Same as & above |
| \| | OR (element-wise) | TRUE\|TRUE is TRUE<br>TRUE\|FALSE is TRUE<br>FALSE\|TRUE is TRUE<br>FALSE\|FALSE is FALSE |
| \|\| | OR (single comparison) | Same as \| above |
| ! | NOT | !TRUE is FALSE<br>!FALSE is TRUE |

```
> FALSE||((T&&TRUE)||FALSE)
[1] TRUE
> !TRUE&&TRUE
[1] FALSE
> (6<4)||(3!=1)
[1] TRUE
```

# CHARACTERS

**Character strings are indicated by double quotation marks, ". To create a string, just enter text between a pair of quotes**

```
> Hi <- "This is a character string!"
> Hi
[1] "This is a character string!"
> length(x=Hi)
[1] 1
> nchar(x=Hi)
[1] 27
> bar <- "23.3"
> bar*2
Error in bar * 2 : non-numeric argument to binary operator
```

# CHARACTERS

## Escape Sequences

| Escape sequence | Result |
|---|---|
| \n | Starts a newline |
| \t | Horizontal tab |
| \b | Invokes a backspace |
| \\ | Used as a single backslash |
| \" | Includes a double quote |

```
> world <- c("awesome","R","is")
> length(x=world)
[1] 3
> world
[1] "awesome" "R"       "is"
> cat(world[2],world[3],"totally",world[1],"!")
R is totally awesome !
> paste(world[2],world[3],"totally",world[1],"!")
[1] "R is totally awesome !"
> cat("here is a string\nsplit\tto neww\b\n\n\tlines")
here is a string
split   to neww

        lines
```

# FACTORS

## Escape Sequences

```
> firstname <- c("Liz","Jolene","Susan","Boris","Rochelle","Tim","Simon", "Amy")
> sex.num <- c(0,0,0,1,0,1,1,0)
> sex.char <- c("female","female","female","male","female","male","male","female")
> sex.num.fac <- factor(x=sex.num)
> sex.num.fac
[1] 0 0 0 1 0 1 1 0
Levels: 0 1
> sex.char.fac <- factor(x=sex.char)
> sex.char.fac
[1] female female female male   female male   male   female
Levels: female male
```

# LISTS AND DATA FRAMES

## List

- creating a list is much like creating a vector. You supply the elements that you want to include to the list function, separated by commas

```
> lst <- list(matrix(data=1:4,nrow=2,ncol=2),c(T,F,T,T),"hello")
> lst
[[1]]
     [,1] [,2]
[1,]    1    3
[2,]    2    4

[[2]]
[1]  TRUE FALSE  TRUE  TRUE

[[3]]
[1] "hello"

> length(x=lst)
[1] 3
> names(lst) <- c("mymatrix","mylogicals","mystring")
> lst
$mymatrix
     [,1] [,2]
[1,]    1    3
[2,]    2    4

$mylogicals
[1]  TRUE FALSE  TRUE  TRUE

$mystring
[1] "hello"
```

# LISTS AND DATA FRAMES

## Data Frames

- creating a list is much like creating a vector. You supply the elements that you want to include to the list function, separated by commas

```
> mydata <- data.frame(person=c("Peter","Lois","Meg","Chris","Stewie"),
+                      age=c(42,40,17,14,1),
+                      sex=factor(c("M","F","F","M","M")))
> mydata
  person age sex
1  Peter  42   M
2   Lois  40   F
3    Meg  17   F
4  Chris  14   M
5 Stewie   1   M
> mydata[2,2]
[1] 40
> mydata$person
[1] "Peter"  "Lois"   "Meg"    "Chris"  "Stewie"
> nrow(mydata)
[1] 5
> ncol(mydata)
[1] 3
> dim(mydata)
[1] 5 3
```

# DATA FRAMES

```
> newrecord <- data.frame(person="Brian",age=7,
+                         sex=factor("M",levels=levels(mydata$sex)))
> newrecord
  person age sex
1  Brian   7   M
```

```
> mydata <- rbind(mydata,newrecord)
> mydata
  person age sex
1  Peter  42   M
2   Lois  40   F
3    Meg  17   F
4  Chris  14   M
5 Stewie   1   M
6  Brian   7   M
```

# DATA FRAMES

```
> funny <- c("High","High","Low","Med","High","Med")
> funny <- factor(x=funny,levels=c("Low","Med","High"))
> funny
[1] High High Low  Med  High Med
Levels: Low Med High
> mydata <- cbind(mydata,funny)
> mydata
  person age sex funny
1  Peter  42   M  High
2   Lois  40   F  High
3    Meg  17   F   Low
4  Chris  14   M   Med
5 Stewie   1   M  High
6  Brian   7   M   Med
```

# CLASS EXERCISE

a. Create and store this data frame as `dframe` in your R workspace:

| person | sex | funny |
|---|---|---|
| Stan | M | High |
| Francine | F | Med |
| Steve | M | Low |
| Roger | M | High |
| Hayley | F | Med |
| Klaus | M | Med |

ST331

(Statistical Computing 3)